

SymPy Bug Report

1 Bug 56: Singularities misses loggamma pole at zero

1.1 Minimal Reproducer

```
from sympy import S, limit, loggamma, symbols
from sympy.calculus.singularities import singularities

x = symbols("x")
s = singularities(loggamma(x), x, S.Complexes)
print("singularities:", s)
print("contains 0:", s.contains(0))
print("loggamma(0):", loggamma(0))
print("limit at 0:", limit(loggamma(x), x, 0, dir="+"))
```

1.2 Observed Behavior

```
SymPy version: 1.14.0
singularities: EmptySet
contains 0: False
loggamma(0): oo
limit at 0: oo
```

The singularity detector reports `EmptySet`, so it claims that no singular point exists in the requested complex domain. This contradicts SymPy's own evaluation of `loggamma(0)` as `oo` and its computed one-sided limit at zero.

1.3 Expected Behavior

The result of the singularity query for `loggamma(x)` over $\mathbb{C}$ should contain 0. More generally, `loggamma(x)` has singularities at the nonpositive integers, so a correct singularity set must include

$$\{0, -1, -2, -3, \dots\}.$$

For the minimal reproducer, the essential correctness requirement is that `s.contains(0)` evaluate to `S.true`.

1.4 Mathematical Explanation

The function `loggamma(z)` is the logarithm of the analytic continuation of $\Gamma(z)$. The gamma function has simple poles at every nonpositive integer:

$$z \in \{0, -1, -2, \dots\}.$$

Consequently, $\text{loggamma}(z)$ diverges at those points. At the representative point $z = 0$, the recorded check gives

$$\text{loggamma}(0) = \infty, \quad \lim_{x \rightarrow 0^+} \text{loggamma}(x) = \infty.$$

Returning $\emptyset$ is therefore not an equivalent representation or a branch-choice issue; it omits an actual singular point. Any singularity set for $\text{loggamma}(x)$ over the complex domain that does not contain 0 is mathematically incomplete.

1.5 Source-Level Diagnosis

The diagnosis identifies the false negative in the implementation of `singularities` around lines 94–108 of `sympy/calculus/singularities.py`. The public call enters `singularities`; the expression remains structurally represented as $\text{loggamma}(x)$ and is not rewritten into $\log(\Gamma(x))$. The detector then scans only a fixed set of elementary structural patterns, such as negative powers and atoms of `log`, `asech`, and `acsch`. Because $\text{loggamma}(x)$ is neither a `Pow` atom nor a `log` atom, none of these scans contributes a singular point and the initially empty set is returned unchanged.

```
for i in expression.atoms(log, asech, acsch):
    sings += solveset(i.args[0], symbol, domain)
...
return sings
```

The relevant special-function behavior is already present around lines 979–984 of the following source file:

`sympy/functions/special/gamma_functions.py`

In that file, `loggamma.eval` evaluates nonpositive integer inputs to ∞ . The problem is that the singularity routine never queries that special-function knowledge and does not have an explicit `loggamma` case. A plausible fix direction, supported by the diagnosis, is to add explicit handling for `loggamma` arguments whose value is a nonpositive integer, or more generally to let the singularity routine consult function-specific singularity hooks instead of relying only on a fixed structural list.

1.6 Additional Failing Instances

The independent verification checks the shifted family $\text{loggamma}(x - a)$ for $a = 0, 1, 2, 3, 4, 5$. In each case, the argument $x - a$ becomes 0 at $x = a$, so the same gamma-function pole implies that $x = a$ must be included in the singularity set. The buggy behavior misses that point throughout the family.

Input / Expression	SymPy behavior	Expected behavior
<code>singularities(loggamma(x), x, C)</code>	does not contain 0	contains 0
<code>singularities(loggamma(x - 1), x, C)</code>	does not contain 1	contains 1
<code>singularities(loggamma(x - 2), x, C)</code>	does not contain 2	contains 2
<code>singularities(loggamma(x - 3), x, C)</code>	does not contain 3	contains 3
<code>singularities(loggamma(x - 4), x, C)</code>	does not contain 4	contains 4
<code>singularities(loggamma(x - 5), x, C)</code>	does not contain 5	contains 5

The independent check also evaluates $\log(\Gamma(\varepsilon))$ numerically for a very small positive ε , confirming the divergent behavior near the pole independently of the symbolic singularity detector.

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and classified this as a new failure mode with no public precedent. The closest related public items concern other `loggamma` behavior, including `lambdify(loggamma(...))`, `loggamma._eval_is_real`, and series or asymptotic behavior, but the artifact identifies no clear duplicate for `singularities(loggamma(x), x, S.Complexes)` returning `EmptySet`. The broader limitation that `singularities` supports only selected structural forms is related, but this exact false negative is individually actionable as a focused regression test and fix.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import S, loggamma, symbols
from sympy.calculus.singularities import singularities

@pytest.mark.parametrize("a", [0, 1, 2, 3, 4, 5])
def test_singularities_loggamma_contains_shifted_zero(a):
    x = symbols("x")
    s = singularities(loggamma(x - a), x, S.Complexes)
    assert s.contains(a) is S.true
```